Manual técnico: Auditor y Auto-Remediador de Seguridad Linux

1. Objetivo y alcance

Este proyecto realiza una inspección rápida de configuraciones críticas habituales en un servidor Linux y emite evidencia legible por personas (Markdown) y máquinas (JSON). Su referencia es un subconjunto educativo de prácticas CIS, llamado aquí “CIS ligero”.

El alcance es deliberadamente reducido: permisos de archivos de cuentas, configuración efectiva de SSH, servicios heredados y firewall. No afirma conformidad con CIS Benchmark ni reemplaza un análisis de vulnerabilidades, una gestión de parches o una política de seguridad.

2. Arquitectura

El ejecutable securityauditor.py tiene cuatro etapas:

1. Descubrimiento: lee metadatos locales y ejecuta comandos sin shell=True.
2. Evaluación: convierte cada resultado en un hallazgo PASS, FAIL, WARN o INFO.
3. Reporte: serializa los hallazgos y el resumen a JSON o Markdown.
4. Remediación opcional: solo se habilita al indicar un control y la confirmación --yes.

No transmite datos ni instala software. El modo de auditoría no requiere privilegios de root, aunque puede tener menor visibilidad sobre algunos recursos.

3. Controles evaluados

3.1 Archivos de cuentas

ID Revisión Criterio de aprobación Riesgo mitigado
|---|---|---|---|
CIS-LITE-001 /etc/passwd UID propietario igual a 0 y sin escritura para grupo/otros Alteración de cuentas y hashes de contraseña
CIS-LITE-002 /etc/shadow UID propietario igual a 0 y sin permisos para otros Exposición de hashes de contraseña

La referencia de modo habitual mostrada por el informe es 0644 para passwd y 0640 para shadow. El segundo control no obliga un grupo: Debian/Ubuntu y Red Hat pueden usar grupos distintos.

3.2 SSH

ID Parámetro efectivo Aprobación Tratamiento
|---|---|---|---|
CIS-LITE-003 PermitRootLogin no Puede remediarse de forma controlada
CIS-LITE-004 PasswordAuthentication no Solo recomendación manual

La consulta se realiza con sshd -T -f /etc/ssh/sshdconfig, que evalúa los valores que aplica

el daemon. Si hay una configuración con sintaxis inválida, el resultado será WARN; corrija primero el archivo y ejecute sshd -t.

3.3 Servicios heredados

Se detecta el estado activo de telnet.socket, rsh.socket, rexec.socket, tftp.socket y xinetd.service mediante systemd. Un servicio activo produce FAIL; uno inactivo, PASS; sin systemd, INFO.

La herramienta no deshabilita servicios: su eliminación puede afectar aplicaciones antiguas que requieren una autorización explícita.

3.4 Firewall

- Si UFW existe, la comprobación aprueba solo si ufw status informa Status: active. Un error del comando produce WARN; una salida válida de firewall inactivo produce FAIL.
- Si UFW no existe pero iptables sí, aprueba si la política por defecto de INPUT es DROP o REJECT.
- Si no existe ninguno, informa WARN.

Una política restrictiva por sí sola no demuestra que las reglas sean correctas. En particular, esta versión no analiza nftables, firewalld ni reglas IPv6.

4. Ejecución y reportes

```bash
python3 securityauditor.py --format json --output reports/host.json
python3 securityauditor.py --format markdown --output reports/host.md
```

Campos principales del JSON:

- generatedat, host, platform: contexto de la ejecución.
- summary: número de resultados por estado.
- findings: ID, título, severidad, evidencia, recomendación y si es remediable.

Los hallazgos tienen estas interpretaciones:

Estado	Significado	Acción esperada
PASS	El criterio revisado se cumple	Conservar como evidencia
FAIL	Se detectó una configuración insegura	Revisar, aprobar y corregir
WARN	No se cumple una recomendación o no se pudo concluir	Analizar el contexto
INFO	El control no puede evaluarse con la plataforma actual	Evaluar manualmente

El programa usa 0 si no detecta fallos, 1 si hay al menos un FAIL y 2 si falla una remediación.

5. Remediación y protección contra bloqueos

No existe una opción “arreglar todo”. Se debe elegir un control y confirmar el cambio:

```bash
sudo python3 securityauditor.py --remediate permissions --yes
sudo python3 securityauditor.py --remediate ssh-root-login --yes
```

permissions establece el propietario UID root (sin cambiar el grupo de la distribución), asigna 0644 a /etc/passwd y 0640 a /etc/shadow. ssh-root-login rechaza configuraciones con Match o Include para exigir revisión manual de las excepciones. En configuraciones simples crea una copia privada con modo 0600 y marca temporal con microsegundos, aplica PermitRootLogin no al principio y ejecuta sshd -t y sshd -T. Si falla la validación o el valor efectivo no es no, restaura el contenido original. No recarga el servicio: el operador debe recargarlo y probar una nueva conexión antes de cerrar su sesión existente.

La auditoría con sshd -T consulta valores globales; no evalúa todas las combinaciones condicionales de Match mediante -C. Un PASS global no garantiza el mismo resultado para cada usuario o dirección de origen.

No se cambia automáticamente PasswordAuthentication: una configuración incorrecta de llaves puede bloquear al operador. Antes de desactivarla, mantenga una segunda sesión abierta, confirme que una cuenta no-root accede por llave y valide la configuración con sudo sshd -t.

Tampoco se aplican reglas de firewall ni se deshabilitan servicios porque esos cambios requieren conocer puertos, dependencias y ruta de acceso de cada host.

6. Operación en Vagrant / home lab

Ejemplo de provisión que solo genera evidencia:

```bash
sudo install -m 0755 /vagrant/securityauditor.py /usr/local/sbin/securityauditor.py
sudo mkdir -p /var/log/security-auditor
sudo python3 /usr/local/sbin/securityauditor.py \
--format json --output /var/log/security-auditor/initial.json || true
```

El || true evita que un hallazgo de endurecimiento corte el provisionamiento; el reporte sigue disponible para revisión. Para producción, conecte el JSON a su sistema de inventario, SIEM o pipeline, y establezca una política explícita para tratar resultados FAIL.

7. Buenas prácticas posteriores

1. Haga copia o snapshot de la VM antes de cambiar SSH/firewall.

2. Pruebe en una VM equivalente antes que en producción.
3. Documente excepciones y su fecha de revisión.
4. Compruebe el acceso tras cada modificación.
5. Complete la evaluación con actualizaciones, MFA/bastion, logs, backups, SELinux/AppArmor, nftables/firewalld y escáneres de vulnerabilidades.

8. Generación del PDF

MANUAL.pdf es la versión imprimible de este documento. Para regenerarlo con Python:

```bash
python3 tools/buildpdf.py docs/MANUAL.md docs/MANUAL.pdf
```

El script utiliza solo la biblioteca estándar de Python y genera un PDF básico, por lo que no requiere Pandoc ni conexión a Internet.